Rejoignez la séance

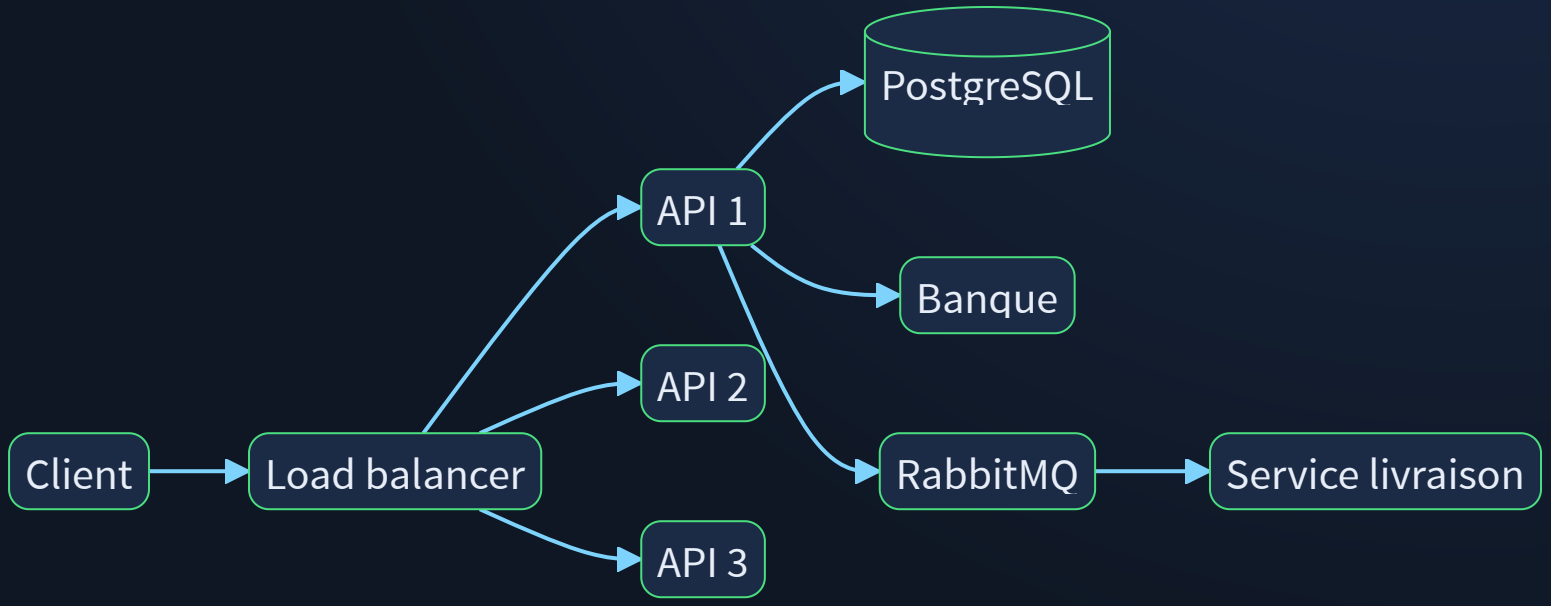
Scannez le QR code ou entrez le code sur votre téléphone.

COURS

Observabilité

Comprendre ce qui se passe en production

FraisDirect en production



Vendredi, 19 h

1. Les clients se plaignent : une commande prend 10 secondes
2. Le tableau de bord : tous les serveurs sont verts
3. Les logs : 3 instances, 2 millions de lignes
4. Par où commencer ?

Au programme

1. Monitoring, alerting, logs, **observabilité** : qui fait quoi
2. Bien écrire ses **logs**
3. Mesurer avec des **métriques**
4. Suivre une requête avec des **traces**

ACTE 1

Qui fait quoi ?

Monitoring, alerting, logs, observabilité

Le monitoring

Vérifier à intervalle régulier une assertion

Un test unitaire, mais en production

- ⊕ Détecte les pannes qu'on sait prévoir : serveur arrêté, disque plein, réseau coupé
- ⊖ Ne détecte pas les pannes inconnues
- ⊖ N'aide pas à résoudre un bug

L'alerting

- Prévenir quelqu'un quand une panne est détectée
- Permet de réagir vite
- Ne dit pas pourquoi

Les logs

- ⊕ Suivre une action en particulier : la commande 42
- ⊖ Pas de vision globale : combien de commandes lentes ? depuis quand ?

L'observabilité

Tout le reste

- ⊕ Comprendre une panne, même inconnue
- ⊕ Faciliter sa résolution
- ⊕ Être sûr qu'on en est sorti
- ⊖ Ne remplace pas le monitoring, les logs et l'alerting

En résumé

Question	Outil
Est-ce que ça marche ?	Monitoring
Qui doit réagir ?	Alerting
Que s'est-il passé pour cette requête ?	Logs
Pourquoi ça ne marche pas ?	Observabilité

QUESTION

Le site ne répond plus à 3 h du matin. Envoyer un SMS à la personne d'astreinte, c'est le rôle…

A. du monitoring

B. de l'alerting

C. des logs

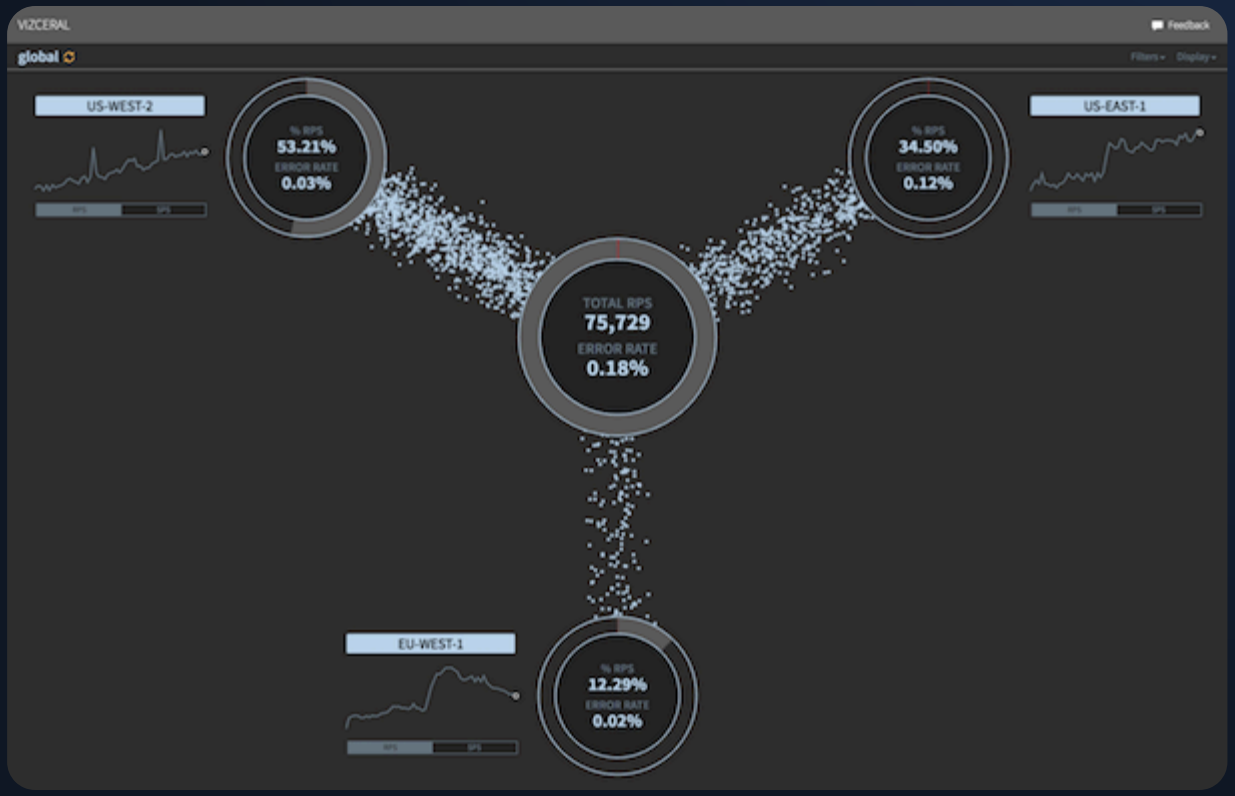
D. de l'observabilité

Pourquoi maintenant ?

- Des systèmes de plus en plus complexes : FraisDirect a déjà 6 composants
- Des disponibilités de plus en plus hautes

Disponibilité	Panne autorisée par an
99 %	3,65 jours
99,9 %	8 h 45
99,99 %	52 minutes

Un exemple : Vizceral



OpenTelemetry

Un standard pour produire les signaux

Signal	Répond à
Logs	Que s'est-il passé ?
Métriques	Combien, à quelle vitesse ?
Traces	Par où est passée la requête ?
Baggage	Quel contexte transporter avec elle ?

ACTE 2

Les logs

Écrire des logs qu'on pourra relire

Le bon niveau

Niveau	Exemple FraisDirect
`ERROR`	La base ne répond plus, la commande est perdue
`WARN`	La banque a répondu en erreur, on réessaie
`INFO`	Commande 42 créée
`DEBUG`	Détail du calcul des frais de livraison
`TRACE`	Chaque ligne du panier

QUESTION

Le paiement d'une commande échoue définitivement : la banque refuse après trois essais. Quel niveau ?

A. `DEBUG`

B. `INFO`

C. `WARN`

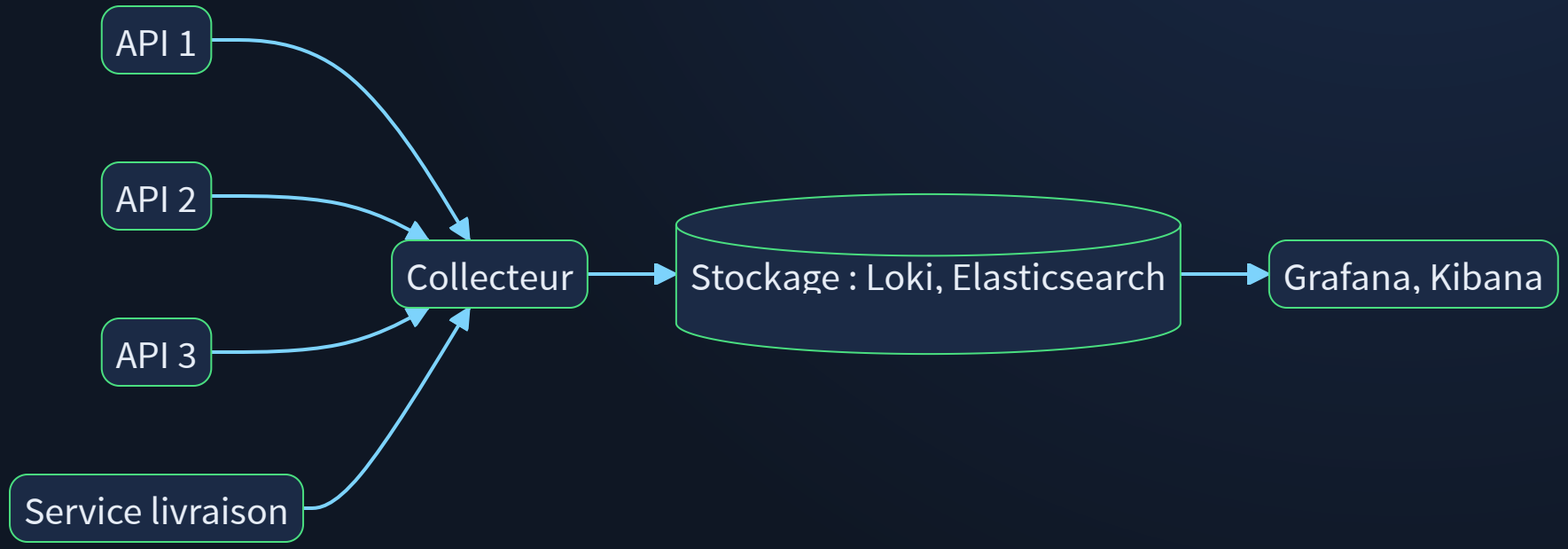
D. `ERROR`

Un log utile

```
1 log.info("Commande créée");
2
3 log.info("Commande {} créée pour le client {}, {} centimes",
4         commande.id(), commande.client(), commande.montant());
```

- Toujours l'identifiant de ce qu'on manipule
- Logguer les **écritures** : création, modification, suppression
- Jamais de mot de passe, de token ou de numéro de carte

Agréger les logs



Un seul endroit pour chercher, au lieu de quatre serveurs

ACTE 3

Les métriques

Des chiffres sur l'application, dans le temps

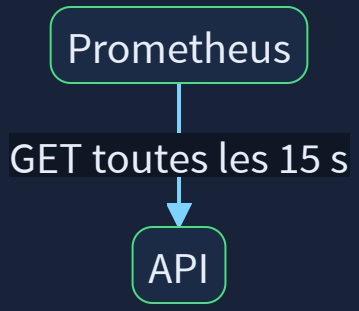
C'est quoi une métrique ?

Une mesure de quelque chose dans l'application

- Un nombre d'actions : commandes passées
- Une durée : temps d'une commande
- Une ressource réservée : connexions à la base

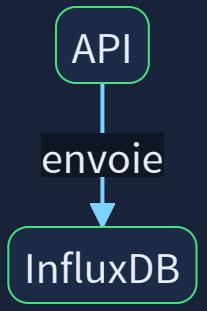
Récupérer les métriques

Scraping



Le système interroge l'application

Push



L'application pousse ses mesures

Ajouter des métriques

- ⛔ À la main, avec un endpoint maison : mauvaise idée
- ➕ Avec **Micrometer** : une façade, comme SLF4J pour les logs
- ➕ Compatible avec Prometheus, InfluxDB, Datadog…

POUR LE PROJET

Étape 1 · Les dépendances

`build.gradle`

```
implementation "org.springframework.boot:spring-boot-starter-actuator"
implementation "io.micrometer:micrometer-registry-prometheus"
```

POUR LE PROJET

Étape 2 · Exposer l'endpoint

`application.yaml`

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
```

POUR LE PROJET

Étape 3 · Vérifier

GET /actuator/prometheus

```
# TYPE http_server_requests_seconds summary
http_server_requests_seconds_count{method="POST",uri="/commandes"} 1532
http_server_requests_seconds_sum{method="POST",uri="/commandes"} 412.7
# TYPE jvm_memory_used_bytes gauge
jvm_memory_used_bytes{area="heap"} 1.43E8
```

Les types de mesure

Type	Exemple FraisDirect
Compteur (Counter)	Commandes passées
Durée (Timer)	Temps de passage d'une commande
Jauge (Gauge)	Messages en attente dans RabbitMQ
Distribution (Distribution summary)	Montant des paniers

QUESTION

On veut suivre le nombre de livreurs disponibles en ce moment. Quel type de mesure ?

A. Compteur

B. Durée

C. Jauge

D. Distribution

Mesurer une commande : mauvaise façon

```java
public void passerCommande(Commande commande) {
    commandesPassees.increment();
    chrono.record(() -> {
        try {
            verifierStock(commande);
            payer(commande);
            publierEvenement(commande);
        } catch (RuntimeException e) {
            commandesEchouees.increment();
            throw e;
        }
    });
}
```

Mieux : séparer métier et mesure

```java
public interface CommandeService {
    void passerCommande(Commande commande);
}

public class CommandeServiceImpl implements CommandeService {
    public void passerCommande(Commande commande) {
        verifierStock(commande);
        payer(commande);
        publierEvenement(commande);
    }
}
```

Mieux : la délégation

```java
public class CommandeServiceMesure implements CommandeService {
    private final CommandeService delegue;

    public void passerCommande(Commande commande) {
        commandesPassees.increment();
        chrono.record(() -> {
            try {
                delegue.passerCommande(commande);
            } catch (RuntimeException e) {
                commandesEchouees.increment();
                throw e;
            }
        });
    }
}
```

Mieux : un aspect

```java
@Aspect
@Component
class CommandeMetriques {
    @Around("execution(* *..CommandeService.passerCommande(..))")
    Object mesurer(ProceedingJoinPoint appel) throws Throwable {
        commandesPassees.increment();
        Timer.Sample chrono = Timer.start(registry);
        try {
            return appel.proceed();
        } catch (Throwable e) {
            commandesEchouees.increment();
            throw e;
        } finally {
            chrono.stop(registry.timer("commande.duree"));
        }
    }
}
```

Encore plus simple : `@Timed`

```java
@Timed("commande.duree")
public void passerCommande(Commande commande) {
    verifierStock(commande);
    payer(commande);
    publierEvenement(commande);
}
```

Micrometer fournit l'aspect tout fait

Tagguer les métriques

```
registry.counter("http.appels", "path", path).increment();
```

- Le nombre d'appels, c'est bien
- Le nombre d'appels **par endpoint**, c'est mieux
- Jamais de valeur sans limite : pas d'identifiant de client en tag

Tagguer toutes les métriques

```
registry.config().commonTags("instance", nomDuServeur);
```

Avec le load balancer : quelle instance est lente ?

ACTE 4

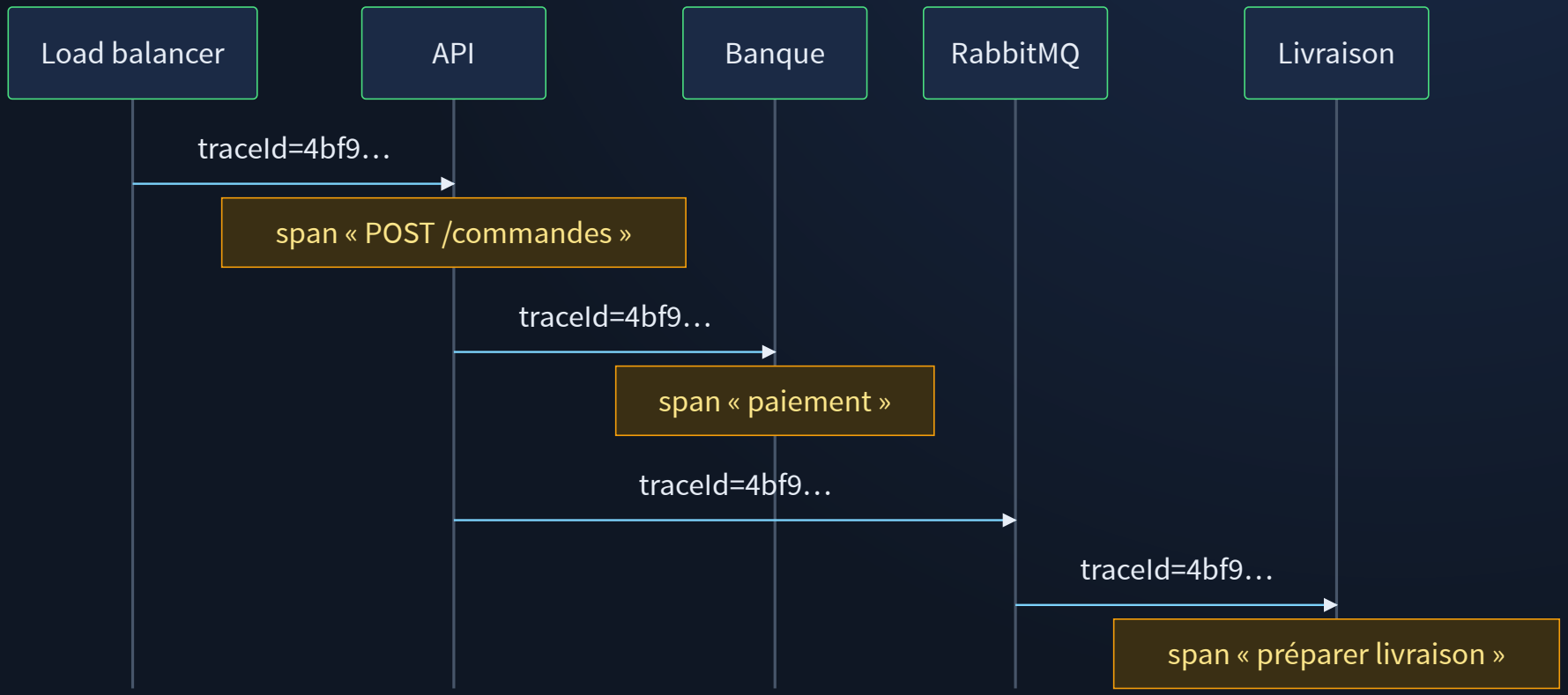
Les traces

Suivre une requête de bout en bout

Le principe

1. Quand une requête arrive, on lui donne un **traceId**
2. On fait suivre le traceId pendant toute la requête, d'un service à l'autre
3. On découpe le traitement en **spans** : une étape, avec un début et une fin

Le traceId voyage



Une trace

`POST /commandes` : 1 240 ms



QUESTION

Vendredi 19 h : la trace montre 1 030 ms sur 1 240 passées à attendre la banque. Que faire en premier ?

A. Ajouter une quatrième instance de l'API

B. Ajouter un index en base

C. Regarder du côté de la banque

D. Redémarrer RabbitMQ

Les traces, à quoi ça sert ?

- Relier les lignes de log d'une même requête
- Voir les communications entre les services
- Corréler les logs et les métriques
- Trouver les points de lenteur d'une requête

Avec Spring Boot

```
implementation "io.micrometer:micrometer-tracing-bridge-otel"
implementation "io.opentelemetry:opentelemetry-exporter-otlp"
```

Le traceId apparaît tout seul dans les logs

```
INFO [api,4bf92f3577b34da6,00f067aa0ba902b7] Commande 42 créée
```

À retenir

- Le monitoring dit si ça marche, l'observabilité dit pourquoi
- Des logs avec le bon niveau et des identifiants, agrégés
- Des métriques avec Micrometer, séparées du code métier
- Des traces pour suivre une requête entre les services